

The Noita Eye Messages

Status report: what has been established, what has been ruled out, and where the cipher stands — 28 September 2026

Summary

Noita hides nine messages written in "eye" glyphs. This project has rebuilt the ciphertext from the game's own pixels, confirmed it byte for byte against the copy embedded in the game executable, and then tested one family of cipher mechanisms after another against it. Every test is gated: before any verdict on the real messages is read, the same instrument must correctly crack or correctly reject synthetic messages whose mechanism is known.

The cipher is **not solved**. What the work has produced is a tight description of what the mechanism must be, and a long list of what it cannot be:

- **Settled.** The ciphertext itself: 9 messages, 1,036 symbols over an 83-letter alphabet, with a unique valid reading order. The game files contain the ciphertext and nothing else — no decoder, no key, no hidden tenth message.
- **Ruled out.** Monoalphabetic and periodic ciphers; every "cut the deck" / cyclic-group walk; walks that skip on collision; a period-4 additive key; row-locked re-keying; a general progressive rotor; the community's large-state deck-shuffle autokey.
- **Required of any answer.** A small hidden state (order 10–20), long exact isomorphs whose letter-mappings do not commute, zero adjacent repeated letters, a sharp excess of repeats four apart, and the ability to fall back into step after short differing stretches.
- **Leading candidate.** A group autokey over the 3,403-element affine group C83:C41 ("gak41"), the one group the community's classification leaves standing. It reproduces 5 of the 7 signatures, more than anything else tried. This week's direct test of it (Step F) came back **inconclusive**: the real data fits, but is far too sparse to pin the hidden alphabet down.

The main open question is no longer "which cipher family?" so much as "is there enough ciphertext to decide?" — the isomorph evidence alone appears to under-determine the answer. The game is Finnish, so the plaintext may well be Finnish rather than English; every future plaintext test must score both.

1. The ciphertext

Each message is a grid of eyes whose pupils point in one of five directions. Grouped in triangles of three eyes, they give base-5 trigrams. The transcription (`tools/transcribe.py`) locates all 3,108 eyes by exact template matching, and proves by exhaustion that exactly one of the 36 possible digit orders makes the trigram values cover 0–82 without gaps. That order is the community's, reached independently.

`tools/exesearch.py` then found the same data inside the current `noita.exe` (file offset 0x21e1bb, packed as base-7 digits in 64-bit constants). The extracted stream of 3,194 characters equals the pixel transcription exactly. That closes the provenance chain: nothing was lost or misread, and there is no further ciphertext in the binary.

Fact	Value
Messages	9 (5 East, 4 West), 3,108 eyes, 1,036 letters
Alphabet	83 symbols, 0–82 contiguous; reading order unique of 36
Letter frequencies	Flat: index of coincidence 0.0128 vs 0.0120 uniform (English text $\approx$ 0.066, Finnish $\approx$ 0.07+)
Shared openings	All 9 share positions 1–2; {east-1, east-2, west-1} share 1–24; {east-4, east-5, west-4} share 1–20
First symbol	Different in all 9 messages — yet position 1 onward is shared, so the first symbol sits outside the cipher's running state

Fact	Value
Isomorphs	18 maximal pairs: the same pattern of repeats recurs at different places (longest 33 letters), strong evidence of repeated plaintext under a changed key

2. The fingerprint any mechanism must reproduce

Seven measurable signatures, each reproduced from the raw data and each measured on synthetic ciphers by the same code, form the test that every candidate mechanism is scored against.

Signature	Real corpus	Why it matters
Long exact isomorphs	up to 33 letters	The letter-mapping between two occurrences of a phrase stays constant for 20–33 letters
Not a translation	walk_dim = 1	No single alphabet order turns every mapping into a shift — kills all "cut the deck" walks
Mappings do not commute	0.38	Composing two mappings depends on order: the key moves through a non-abelian structure
Zero adjacent doubles	0 vs ≈ 12 expected	The mechanism can never emit the same symbol twice in a row
Period-4 recurrence	2.6 $\times$ at distance 4	Something cycles with period four
Resynchronisation	rate ≈ 0.065	Messages that differ briefly fall back into step — the hidden state is small
Flat frequencies	IoC 0.0128	No simple substitution

Figure 1. Repeated symbols by distance, relative to chance



Observed count of equal symbols d positions apart, divided by the mean over 400 within-message shuffles. Distance 1 never happens, distance 2 is suppressed, and distance 4 is more than double chance (26 vs 11.6, $z \approx 4.4$).

3. What has been ruled out

Each line is a separate tool with its own synthetic controls. A "refuted" verdict was accepted only after the instrument had shown it could crack a cipher of that family when one was really there.

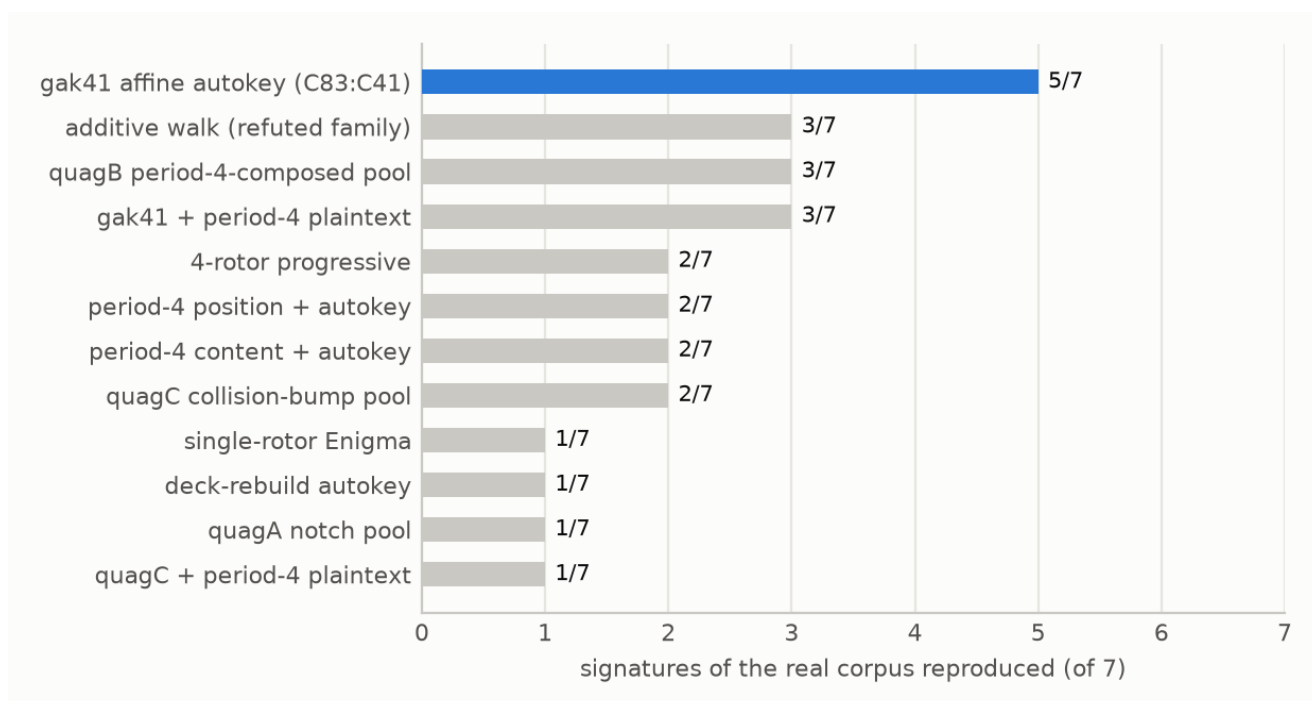
Hypothesis	Verdict	Deciding evidence
Simple substitution / periodic polyalphabetic	Refuted	Flat frequencies, no periodicity, isomorphs
Additive walk ("cut deck", cyclic group C83)	Refuted	Jump-safe core system has no injective solution (sigma_web)
Walk with collision-skip, any jump density	Refuted	Sparse skips would crack; dense skips destroy the isomorphs
Any cyclic group of order 59–127	Refuted	Core system inconsistent at every prime in range
Re-keying locked to the physical eye rows	Refuted	The 33-letter isomorph runs straight across a row boundary

Hypothesis	Verdict	Deciding evidence
Period-4 additive key index	Refuted	Isomorph offsets spread over all residues mod 4
Pure period-4 keystream	Refuted	Would force every isomorph to identity mapping at offset $\equiv 0 \pmod 4$
Large-state deck-shuffle autokey (S83)	Refuted	Could not re-synchronise; the corpus does so four times
General progressive rotor ($\sigma = T^{\delta}$)	Not supported	0 of 4 independent edges fit (small sample)
Plaintext crib on the shared openings	Closed	No isomorph touches the openings, and nothing could validate a guess
Decoder or key shipped with the game	Refuted	Full keyword and pixel sweep of all 14,745 data files, plus the executable

4. The candidate tournament

Candidate mechanisms are implemented as small encryptors, run over a shared synthetic corpus (repeated phrases, shared openings, realignments), and scored on how many of the seven real signatures they reproduce.

Figure 2. Signatures reproduced by each candidate



Merged harness, seed 1. Over a 20-seed sweep gak41 keeps a median of 5/7 and reproduces the "not a translation" signature in 14 of 20 runs — the only candidate ever to do so.

gak41 is a *group autokey*: each plaintext letter multiplies the hidden state by an affine map $x \rightarrow bx + a \pmod{83}$ whose multiplier b is restricted to the order-41 subgroup, and the output is a fixed relabelling of the state's translation part. Nonzero a makes adjacent doubles impossible, and two occurrences of a phrase share the running product, which gives exactly the constant, non-commuting mappings the real isomorphs show. Its misses are the resync rate and the period-4 recurrence.

A correction made this week. Two lines of work had run in parallel on different machines and were merged on 28 September. One of them had concluded that the period-4 excess could come from the plaintext (for example a list with a separator every fourth symbol). Re-run in the merged harness and swept over 20 seeds, that effect appears in only 5 of 20 runs, and adding it to gak41 breaks gak41's best signature. The claim was withdrawn: period-4 remains unexplained.

5. Step F: a direct test of gak41

If gak41 is right, every isomorph mapping, written in the hidden alphabet order π , is an affine map $\pi(b) = \beta \cdot \pi(a) + \alpha$ with β a nonzero square mod 83 (the order-41 subgroup is exactly the quadratic residues). No earlier phase had tested this: they all allowed only shifts ($\beta = 1$). The new tool, `tools/affinefit.py`, searches the 41 possible β values for each block of evidence, keeping a linear system for π over GF(83), and prunes any branch that becomes inconsistent or forces two letters onto the same value.

Control / run	Result
gak41 synthetic (274 aligned pairs)	PASS — one assignment found, the true one; π recovered exactly up to relabelling
Additive walk ($\beta = 1$ case)	PASS — accepted
Fresh random alphabets (319 pairs)	PASS within budget — no fit found (exhaustive proof would take hours)
Real corpus, all blocks	No fit — but only because of two short stretches, explained below
Real corpus, those stretches removed	Fits, 2,050+ assignments; null space of dimension 8; only the 2 reference letters pinned

Two method repairs came out of this. First, the "witness span" rule that earlier phases used to decide which positions are safe to trust lets through a few positions where two plaintexts have not yet converged, because the window-finder favours chance matches at its edges. Requiring two independent witnesses fixed it (contaminated blocks on known-truth data fell from 9 of 377 to 0 of 420). Second, east-1 and west-1 are identical except for two four-symbol patches (positions 25–28 and 33–36). There the hidden state leaves and then returns exactly, which no witness can detect, and those patches were the entire source of the "no fit" result.

Verdict: inconclusive. gak41 is neither refuted nor recoverable from the isomorphs. The real mapping web has about 122 usable letter pairs over 60 letters; the synthetic test that pinned π uniquely had 274. With this little evidence, many different affine alphabets explain it equally well.

6. Current theories

Ranked by how well they fit what is known. None is established.

Theory 1 — a small-state, non-commuting autokey (leading)

The key state is updated by each plaintext letter through a non-commutative structure (a group or something group-like), the output is a fixed function of that state, and the first symbol of each message sits outside the chain as a header. This single shape explains the constant, non-commuting isomorph mappings, the zero doubles and the shared openings. gak41 is the concrete, testable member of this family and fits best so far.

Main tension: resynchronisation. When two messages differ for a few letters, a group autokey only falls back into step if the two differing snippets happen to have the same group product. In a group of 3,403 elements that is rare, yet the corpus resynchronises four times, and Step C bounds the effective state at around 15. Either the plaintext differences are special (for example rearranged letters that commute, or number changes in a list), or the state really is smaller than gak41's.

Theory 2 — the state is smaller than gak41, with a period-4 component

The resync rate and the distance-4 excess both point at a state of order 10–20 that has a four-step cycle inside it. Nothing tested so far has produced a crisp period-4 while keeping the long exact isomorphs: making the four-step cycle depend on absolute position destroys the isomorphs, and making it depend on the plaintext blurs it. A mechanism that gets all three at once has not been found; finding one is the clearest open design problem.

Theory 3 — the evidence is simply too thin (a working caution)

Several lines now end the same way: the 18 isomorph pairs constrain the hidden alphabets too weakly to single one out (Step F, rotorfit, alphacount). It is possible that the nine messages alone cannot determine the key, whatever the mechanism. That would match the community's long-standing "we need more messages" caution, and the executable search has now shown there are no more.

On the plaintext language

Noita is by Nolla Games, a Finnish studio. The plaintext may be Finnish, and 83 symbols is far more than either alphabet, so it may also include capitals, digits or punctuation. Finnish has a higher index of coincidence than English and distinctive letter statistics (frequent a, i, t, n, e, s; the letters ä and ö), so a Finnish-aware score is now part of the standing test for any candidate decryption.

7. Recommended next steps

- **Quantify the resync tension for gak41.** Simulate gak41 with realistic Finnish and English plaintext and structured edits (swapped words, changed list numbers) and measure how often it falls back into step. If no plausible edit pattern reaches the observed rate, gak41 loses its main support.
- **Use the resync points as equations.** Each exact return of the state is a constraint on the mechanism (equal group products) that the isomorph search does not use. Adding them may tighten Step F's eight-dimensional freedom.
- **Look for a period-4 mechanism that keeps exact isomorphs.** The tournament shows the shape of the gap precisely; the next candidates should be designed against it.
- **Build the Finnish plaintext model** (letter and bigram statistics, an index-of-coincidence target) so any future decryption attempt is judged in both languages.
- **Housekeeping.** Clean up the stale copy on the third machine, and make the alphabet-count report's output order deterministic.

Appendix: tools and reproducibility

Everything is in the private repository TheWhyteWolf/noiteyes. Each tool reads only data/messages.json (or the game files), writes a JSON report to data/, and exits non-zero if any control gate fails.

Tool	Purpose
transcribe.py	Pixels to ciphertext; proves the unique reading order
exesearch.py	Extracts the eye data from noita.exe; confirms the transcription byte for byte
wak_unpack.py, wak_sweep.py, eye_mural_scan.py	Reads and exhaustively searches the game's data archive
isomorphs.py, pi_solver.py	Isomorph catalogue; first walk-model test
sigma_web.py	Jump-safe mapping diagnostics; refutes the whole walk family
rowphase.py, phase4.py	Row-locking and period-4 tests
resync.py	Bounds the hidden state size from resynchronisations
tournament.py	Scores candidate mechanisms against the fingerprint
rotorfit.py	Tests the general progressive rotor
alphacount.py	Minimum number of distinct alphabets the mappings force
affinefit.py	Step F: affine alphabet solver for the gak41 hypothesis

Working principle throughout: trust nothing that has not been reproduced from primary data, and believe no decryption until it passes a gauge-invariant statistical test and reads consistently across all nine messages.